

FabriQ: An End-to-End Fabric Defect Detection System Using YOLOv8 and a Web Dashboard

Technical Report (Draft for Conference / Journal Paper)

Authors: [Your Name(s)] Affiliation: [Your Institution] Date: May 2026 Version: 1.0

Abstract

Fabric defect detection is critical for quality assurance in textile manufacturing. Commercial inspection systems are often costly and poorly suited to small and medium enterprises (SMEs). This report describes **FabriQ**, an integrated software stack that combines **YOLOv8-based object detection**, a **Flask REST API**, a **React (Vite) dashboard**, **Firebase (Firestore + Storage)** for persistence, and **live camera streaming** with server-side inference suitable for deployment scenarios ranging from laptop prototyping to edge devices such as the NVIDIA Jetson Nano with industrial or Arducam sensors. The system supports image and video upload, frame-wise annotated video output, real-time MJPEG streams with bounding boxes, and analytics derived from stored defect records. We outline dataset preparation, training considerations for CPU and GPU environments, system architecture, deployment options, and recommended MLOps practices. Quantitative benchmarks should be inserted by the authors after formal evaluation on their held-out test set.

Keywords: fabric defect detection, YOLOv8, computer vision, Flask, React, Firebase, MLOps, edge deployment, textile industry

1. Introduction

Textile production involves continuous material movement on rollers, making automated visual inspection valuable for reducing waste and maintaining grade consistency. Traditional manual inspection is subjective and does not scale. Deep learning detectors, particularly single-stage detectors such as YOLO variants, offer a practical balance between accuracy and inference latency.

FabriQ targets affordability and deployability for contexts such as Pakistan's SMEs by providing:

1. A standardized pipeline from messy folder structures to YOLO-compatible datasets.
2. Training scripts for classification and detection (including pseudo-annotations where full box labels are unavailable initially).
3. A web application for supervisors: detection UI, dashboard metrics, analytics charts, and exportable reports.
4. Cloud persistence via Firebase for defects and optional annotated imagery.
5. Live inspection via backend camera capture and streamed annotated frames.

This document serves as a **technical report** and **paper scaffold**: methodology, architecture, and implementation details are included; empirical results tables should be completed from your experiments.

2. Related Work and Background

Fabric defect detection has been addressed with handcrafted features, classical machine learning, and convolutional neural networks. Recent work favors deep detectors and segmentation for fine defects. YOLO families (v5–v8/v11) are widely used in industrial vision for real-time bounding-box prediction.

Positioning: FabriQ emphasizes **operational completeness** (API + UI + storage + streaming + CI skeleton) rather than introducing a novel backbone. The contribution suitable for a paper draft is **system integration, deployment path to edge cameras**, and **repeatable MLOps hooks** (dataset tooling, model path configuration, GitHub Actions CI).

3. Problem Statement

Inputs: Images or video of fabric; optionally a live camera attached to the inference machine (laptop, workstation, or Jetson).

Outputs: Bounding boxes and class labels for defects; annotated previews; optional stored records for analytics.

Constraints: SMEs require low cost, understandable UX, and ability to swap trained weights without rewriting application code. Edge deployment imposes limits on model size, thermal budget, and camera interfaces (USB, CSI).

4. Dataset and Preparation

4.1 Organization Pipeline

Raw images often reside in heterogeneous folders. A Python utility (``organize_dataset.py``) maps folder and filename cues to a fixed **20-class** taxonomy used during early dataset consolidation, with fuzzy mapping from common terms (e.g., hole → knit hole) and standardized filenames plus **80/20 train/validation** split under ``FabriQ_Final_Dataset/train/images`` and ``val/images``.

4.2 Detection-Oriented Dataset

Where only class folders exist, **pseudo-annotations** (``create_pseudo_annotations.py``) generate full-image YOLO boxes (``class_id 0.5 0.5 1.0 1.0``) to bootstrap detection training; authors should replace these with **real box annotations** as labeling matures.

4.3 Schema Evolution

The deployed application uses a **four-class** defect schema aligned with backend inference:

`- `hole` - `objects` - `oil_spot` - `thread_error``

Firestore-facing normalized labels map to ``hole``, ``stain``, ``broken_thread``, and ``misweave`` for dashboard consistency. Authors must keep **`CLASSES`** in ``backend/app.py`` aligned with the **order of classes in the trained YOLO weights** (Ultralytics uses class indices).

5. Model Training

5.1 Framework and Model

Ultralytics YOLOv8 is used for detection training (``train_yolo_detection.py``). Device selection adapts to CUDA when available; CPU training remains supported with reduced batch size and epochs.

5.2 Training Environment Notes

Training has been exercised in local Windows environments and in notebook hosts (e.g., Kaggle). Practical issues addressed in the project include:

- **NumPy / OpenCV / PyTorch** version alignment (e.g., NumPy ≤ 2 with certain torch builds). - **Corrupt JPEG** handling during training data ingest. - **PyTorch 2.6** `weights_only` loading: safe globals for Ultralytics `DetectionModel` where applicable.

5.3 Evaluation (Placeholders for Paper)

Authors should report **precision, recall, mAP@0.5, mAP@0.5:0.95**, inference FPS, and confusion trends on a **fixed test split**. Include hardware (GPU model, Jetson revision), image resolution, and confidence threshold.

6. System Architecture

6.1 High-Level Components

Layer	Technology	Role
Frontend	React, Vite, Material UI	UX, charts, tables, auth client
Backend	Flask, OpenCV, Ultralytics	Inference, uploads, live stream
Persistence	Firebase Firestore + Storage	Defect records, optional images
Ops	Docker Compose (optional), GitHub Actions	Packaging and CI

6.2 Inference Paths

1. **Upload (image/video)**: multipart POST to `/api/detection/analyze`; JSON returns defects, optional base64 annotated image, annotated video URL for uploaded video. 2. **Live stream**: `POST /api/live/start` opens `cv2.VideoCapture` at configurable index; `GET /api/live/stream` returns MJPEG with drawn boxes; `GET /api/live/latest` exposes latest frame metadata for optional Firestore sync.

6.3 Authentication

JWT-based login protects API routes (`Authorization: Bearer ...`). Stream URLs may pass token as query parameter where browser `<img>` cannot set headers.

6.4 Model Selection at Runtime

Priority order (backend):

1. Environment variable `FABRIQ_MODEL_PATH` 2. Relative weights path under `runs/detect/...` 3. Absolute fallback path 4. Pretrained `yolov8n.pt` if custom weights missing

Camera index: `FABRIQ_CAMERA_INDEX` or UI-selected index passed to `/api/live/start`.

7. Frontend Application

7.1 Modules

- **Login** — obtains JWT stored client-side. - **Dashboard** — KPI-style stats and defect distribution fed from Firestore aggregations. - **Detection** — upload, annotated preview, live stream viewer, camera source selector (laptop vs future Arducam/USB indices). - **Analytics / Reports** — trends and tabular export (PDF/CSV patterns).

7.2 Firebase Integration

Client SDK initializes Firestore and Storage; defect writes occur after successful inference (upload path with resilient Storage fallback). Live stream polling can append defect rows when new frame IDs contain

detections.

Security note: Production deployments should move Firebase config to environment variables and tighten Storage/Firestore rules.

8. Deployment

8.1 Local Development

- Backend: Python virtual environment, ``pip install -r backend/requirements.txt`, `python backend/app.py``.
- Frontend: ``npm install`, `npm run dev`` with ``VITE_API_URL`` pointing to backend.

8.2 Docker

``docker-compose.yml`` orchestrates services where configured; align ports and environment variables for model path and API URL.

8.3 Edge (Jetson Nano)

Use ARM-compatible PyTorch/Ultralytics builds; set ``FABRIQ_CAMERA_INDEX`` for CSI/USB camera; monitor thermal throttling; consider TensorRT export for latency (future enhancement).

9. MLOps and Reproducibility

The repository includes:

- ``.gitignore`` excluding large weights, ``runs/``, datasets, and uploads. - **GitHub Actions** workflow for frontend lint/build and backend dependency smoke checks. - ``.MLOPS_GUIDE.md`` describing branching, artifact promotion, and suggested registry options.

Recommended practices:

- Tag releases with model checksum and dataset version. - Store evaluation notebooks and metric tables alongside each release. - Never commit secrets or production JWT secrets.

10. Limitations and Future Work

- Pseudo-label full-image boxes are weak supervision; replace with precise annotations. - Live Firestore sync from stream uses sampled latest-frame metadata; high-frequency logging may require batching or backend-side writers. - Explainability (Grad-CAM, attention maps) and formal RBAC are outlined as extensions. - Jetson TensorRT pipeline and GStreamer zero-copy capture are natural next steps.

11. Conclusion

FabriQ demonstrates a **complete, modular** fabric defect detection stack combining YOLOv8 inference, web UX, cloud persistence, and live inspection suitable for SME workflows and edge rollout. The accompanying codebase and this report provide a foundation for a full paper once empirical metrics and ablation studies are completed.

References (Template — Replace with Formal Citations)

1. Redmon, J., et al. YOLO series (historical). 2. Ultralytics YOLOv8 Documentation. <https://docs.ultralytics.com> 3. Flask Documentation. <https://flask.palletsprojects.com> 4. Firebase Documentation. <https://firebase.google.com/docs>

Appendix A: Key Repository Artifacts

Artifact	Purpose
<code>`organize_dataset.py`</code>	Dataset structuring and splits
<code>`train_yolo_detection.py`</code>	Detection training entry
<code>`create_pseudo_annotations.py`</code>	Bootstrap labels
<code>`backend/app.py`</code>	API, inference, live stream
<code>`frontend/`</code>	Dashboard application
<code>`fabriq_detection_data.yaml`</code>	YOLO data config (if used)
<code>`MLOPS_GUIDE.md`</code>	Operational guidance

Appendix B: Figures to Add in Final Paper

Suggested figures for camera-ready submission:

1. System architecture diagram (frontend ↔ API ↔ model ↔ Firebase). 2. Sample detections (oil spot, hole, thread error) with bounding boxes. 3. Training loss / validation mAP curves. 4. Dashboard screenshots (blurred if proprietary).

End of technical report draft.